

Supplementary Material for Back to the Future: Regressing Readability Features from LLMs

ANONYMOUS AUTHOR(S)

CCS Concepts: • **General and reference** → **Measurement**; • **Software and its engineering** → *Maintaining software*; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: code readability, program comprehension, software metrics, large language models, interpretability

ACM Reference Format:

Anonymous Author(s). 2027. Supplementary Material for Back to the Future: Regressing Readability Features from LLMs. In *Proceedings of the 35th ACM Symposium on the Foundations of Software Engineering (FSE '27)*, July 12–16, 2027, Shenzhen, China. ACM, New York, NY, USA, 21 pages.

A Feature Extraction and Candidate Feature Set

This appendix specifies the extraction rules and complete candidate inventories for the three feature families in the main paper’s approach section.

A.1 Feature Extractor

Each extracted snippet records a normalised string, its source-line position, and one of the types listed in Table 1.

The snippet representation is language aware. Complete Java and Python units are processed using their corresponding syntactic representations, while explicitly declared fragments and C-family code use bounded language-specific lexical extraction. Extracted strings are whitespace-normalised and bounded to prevent unusually large constructs from dominating the embedding computation.

Representative Java examples appear in Table 1. Because one construct can emit multiple snippets, the extracted strings shown in the table are illustrative rather than exhaustive.

Snippet types and occurrence counts are used to construct traditional inventory features. Their normalised strings are embedded and used to construct semantic geometry, clustering, short-identifier context, and comment-alignment features.

A.2 Traditional Candidate Features

The traditional inventory contains 95 features. These measurements are computed directly from the source text and AST-tagged snippets and do not depend on an embedding model. The current feature groups are summarised in Table 2. The counts refer to produced feature columns, excluding dataset identity, task identifier, and readability-label columns.

Each snippet type contributes only the count or ratio measurements defined for that type. For example, literals are represented through the expression–literal balance rather than a total-snippet-normalised literal ratio. The selected *expression–literal density* is the add-one-smoothed log ratio

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE '27, Shenzhen, China

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Table 1. Snippet taxonomy and representative extracted strings.

Type	Definition	Source code	Representative extracted string
IDENTIFIER	Named program entity or reference, such as a variable, field, parameter, class, method, or constant	return count;	count
DECLARATION	Introduction or definition of a program entity, including its name and available type or parameter information	int count = 0;	int_count
CALL	Function, method, or API invocation	items.size();	items.size
CONTROL_FLOW	Construct that determines or changes execution order	if (count > 0)	if_count>0
ASSIGNMENT	Binding or update of a variable, field, array element, or object property	sum += value;	sum+=value
ARITHMETIC	Numerical computation or arithmetic subexpression	total / count	total/count
COMPARISON	Relational, equality, type, or membership predicate	name != null	name!=null
LOGICAL	Boolean composition or negation of predicates	ready && !closed	ready&&!closed
BITWISE	Bit-level operation, shift, or mask	flags & READ_ONLY	flags&READ_ONLY
LITERAL	Constant value appearing directly in source code	return 3;	literal_3
REGEX	Regular-expression component extracted from a string literal used as a regular expression	Pattern.compile("\\d+");	regex_\d, regex_+
IMPORT	Standard-library or external dependency referenced by the code	import java.util.List;	import_stdlib_util_List
UNUSED_IMPORT	Imported dependency not referenced by the code	import java.io.File;	unused_import_stdlib_io_File
COMMENT	Natural-language text embedded in source code	// initialize the cache	comment_initialize_the_cache

$$\log(1 + N_{\text{arithmetic}} + N_{\text{bitwise}} + N_{\text{comparison}} + N_{\text{logical}}) - \log(1 + N_{\text{literal}}).$$

It contrasts compound operations and relations with literal values stated directly in the source.

Table 2. Traditional candidate feature groups.

Feature group	Representative measurements	Count
Text, lexical, and scale	Non-empty lines of code, line-length statistics, indentation, vocabulary size, token count, Halstead volume, comparison and operator density, and byte entropy	13
Visual layout and token distribution	Indentation transitions, long-line ratios, keyword and punctuation density, visual-area ratios, normalised token positions, DFT-based periodicity, per-line token maxima, and aligned visual blocks	32
Snippet inventory and geometry	Snippet count, distinct-snippet ratio, per-line snippet distribution, snippet-character and snippet-token variation, snippet span, expression–literal balance, and the available count or ratio measurements for each snippet type	32
Identifier, member-access, and comment structure	Identifier-length statistics, single-letter and digit-character ratios, identifier subtoken count, member-access density and chain depth, comment–identifier word overlap, and line-level identifier coherence	12
Compression and redundancy	Whole-snippet and line-level zlib ratios, five-line-block LZMA ratios, and cross-line redundancy under zlib and LZMA	6
Total		95

Table 3. Snippet views used for embedding-derived features.

View	Included snippet types
all	All snippet types
only_identifier	IDENTIFIER
semantic_core	IDENTIFIER, DECLARATION, CALL, and LITERAL
structural_core	CONTROL_FLOW, ASSIGNMENT, ARITHMETIC, COMPARISON, LOGICAL, and BITWISE

A.3 Embedding-Derived Candidate Features

The embedding-derived definitions are instantiated independently using nomic-embed-text-v1.5 [14], Qwen3-Embedding-0.6B [24], jina-embeddings-v2-base-code [6], snowflake-arctic-embed-m-v2.0 [23], and voyage-4-nano [22].

Snippet views. Embedding geometry and clustering are evaluated over four deterministic views of the snippet sequence. The views separate complementary kinds of program information without using readability labels.

Each distinct type–string pair is embedded once and retains its source occurrence count as a weight. Transformer outputs are converted to one vector using attention-mask-aware mean pooling. The vectors are subsequently normalised before cosine geometry and clustering features are computed.

For exceptionally large units, feature computation retains at most 512 distinct type–string rows within a view. Selection is deterministic: higher-frequency rows are retained first, with snippet type and extracted string used to resolve ties.

Feature inventory. The embedding-derived inventory contains 102 features for each embedding model. The same feature definitions are instantiated independently for each of the five embedding representations; model-specific matrices are not concatenated.

Table 4. Embedding-derived candidate feature groups.

Feature group	Measurements	Count
Targeted semantic alignment	Comment–identifier cosine alignment, calibrated comment–code relevance, short-identifier prevalence, and mathematical-versus-application context	9
Snippet-view coverage	Fraction of source-weighted snippets represented by the only_identifier, semantic_core, and structural_core views	3
Embedding geometry	Mean cosine distance to the centroid, pairwise cosine-distance dispersion, effective rank where applicable, and variance not explained by the first principal component	13
OPTICS	Unclustered ratio, cluster-size variation, maximum cluster diameter, mean reachability, and pattern mixing where applicable	19
HDBSCAN	Unclustered ratio, largest-cluster ratio, cluster-size variation, maximum cluster diameter, and pattern mixing where applicable	19
Automatically selected K-means	Selected cluster count, silhouette score, largest-cluster ratio, cluster-size variation, and maximum cluster diameter	20
Automatically thresholded agglomerative clustering	Cluster count, largest-cluster ratio, cluster-size variation, maximum cluster diameter, and pattern mixing where applicable	19
Total		102

The exact number of produced features in each family is reported in Table 4. Cluster composition is represented by normalised pattern mixing.

Adaptive clustering. The four clustering procedures use deterministic or data-adaptive configuration rules rather than a fixed-radius DBSCAN sweep. Let n denote the number of distinct embedding rows available for a snippet view.

OPTICS [1] uses cosine distance, $\xi = 0.05$, and

$$\text{min_samples} = \min(n, \max(2, \text{round}(\log_2 n))) .$$

HDBSCAN [4] operates on precomputed cosine distances. Its minimum cluster size and minimum sample count are both set to

$$\min(n, \max(2, \text{round}(\log_2 n))) .$$

K-means [11] evaluates

$$k \in \{2, 3, 4, 6, 8, 12, 16\},$$

subject to the available number of samples and distinct vectors. The value maximising cosine silhouette [17] is selected. Each fit uses ten initializations and a fixed random state.

Agglomerative clustering uses average linkage and cosine distance. Its distance threshold is set to the median positive pairwise cosine distance within the current view, allowing the resulting number of clusters to adapt to the embedding geometry of each source unit.

Targeted semantic measurements. Two targeted feature groups address readability cues that cannot be characterised reliably from surface form alone.

For short identifiers, we first compute

$$r_{\text{short}} = \frac{N_{\{a,b,c,d,e,i,j,k,m,n,x,y,z\}}}{N_{\text{identifier}}},$$

where the numerator counts identifiers belonging to the fixed short-name candidate set. For source units containing at least one such candidate, code segments containing a candidate are compared with fixed mathematical and application-code reference centroids. The semantic context feature is the mean margin

$$m_{\text{short}} = \frac{1}{|S|} \sum_{s \in S} [\cos(e_s, \mu_{\text{math}}) - \cos(e_s, \mu_{\text{application}})],$$

where S is the set of candidate-bearing code segments. The feature records the semantic setting in which those identifiers occur.

Comment-related features are divided into two complementary groups. First, three features measure alignment between comments and identifiers using the mean pairwise similarity, maximum pairwise similarity, and similarity between the weighted comment and identifier centroids. Second, four features characterise the relevance of comments to code. For each comment, we compute its maximum cosine similarity to a non-comment semantic snippet. The resulting similarities are summarised by their weighted mean and minimum.

To identify potentially irrelevant comments without using readability labels, each embedding model receives an independently calibrated similarity threshold. Calibration uses 24 fixed code-comment anchors: eight Java, eight Python, and eight JavaScript examples. Each anchor provides one relevant comment and one deterministic mismatched comment borrowed from another anchor in the same language, producing 48 labelled calibration comparisons. The threshold maximises balanced accuracy and is stored with the embedding-model-specific calibration artefact.

The final two comment-relevance features are the source-weighted fraction of comments below this threshold and their mean positive similarity deficit. The calibration corpus is fixed independently of the six readability benchmarks and is not fitted using human readability scores.

A.4 Causal-LM Predictability Candidate Features

The extended representation adds 47 canonical feature definitions, instantiated independently with three frozen causal code language models: Qwen2.5-Coder-0.5B [9], deepseek-coder-1.3b-base [7], and OpenCoder-1.5B-Base [8]. The models assign teacher-forced probabilities to the observed source, from which the predictability measurements are computed.

Let $p(t_i | t_{<i})$ denote the teacher-forced probability of the observed token t_i . Its surprisal in bits is

$$s_i = -\log_2 p(t_i | t_{<i}).$$

Because a token can cover a different number of source bytes under different tokenisations, most aggregate measurements use bits per byte (BPB). For a source region R ,

$$\text{BPB}(R) = \frac{\sum_{i \in R} s_i}{\text{number of covered UTF-8 source bytes in } R}.$$

Token losses are aligned back to the original source offsets. When a token overlaps only part of a measured region, its surprisal is allocated uniformly across its covered source bytes. Unions of source spans are used for denominators so that overlapping syntactic roles do not count the same byte twice within one measurement.

The complete 47-feature inventory is summarised in Table 5. Comments are deliberately excluded from the generic role-difficulty group: the difficulty of predicting natural-language comment text

Table 5. Causal-LM predictability candidate feature groups.

Feature group	Measurements	Count
Global difficulty	Complete-source perplexity, complete-source BPB, and the mean BPB of the most difficult 20% of local token blocks	3
Typed-unit difficulty	BPB mean, difficult-tail mean, and occurrence-level standard deviation for identifiers, declaration headers, call targets, control-flow headers, assignment right-hand sides, expressions, and literals	21
Local organisation	Block-level BPB standard deviation, tail-to-global ratio, mean and 90th-percentile adjacent-block jumps, longest difficult-block run, and Gini concentration	6
Identifier introduction and reuse	First-occurrence and reuse BPB, difficult reuse tail, positive reuse difficulty increase, within-binding variation, short-identifier BPB, and first-subtoken and continuation-subtoken BPB	8
Contextual support	Longer-context gain for identifiers, short identifiers, call targets, expressions, assignment right-hand sides, and control-flow headers; 10th-percentile identifier gain; and two measures of the predictive information supplied by preceding comments	9
Total		47

does not by itself establish whether the comment helps a reader understand the code. Comments instead contribute through the two code-prediction-gain measurements described below.

The typed-unit spans are extracted from the source syntax rather than from tokenizer strings. Declaration headers exclude bodies and variable initializers; call targets exclude argument expressions; control-flow headers exclude their controlled bodies; and assignment measurements use only the initializer or right-hand-side expression. Literal spans exclude documentation strings. For each role, the mean BPB is weighted by covered bytes, the tail is the mean of the most difficult 20% of occurrences, and the standard deviation measures heterogeneity across occurrences.

Local-organisation features divide the source trace into consecutive 32-token blocks. They distinguish uniformly difficult code from code in which surprisal is concentrated in a few regions or changes abruptly. The difficult-block run uses the within-source 80th percentile and is normalised by the number of blocks.

Identifier-reuse features operate on confirmed parameter and local-variable bindings. Identical spellings in different scopes are not merged. The first occurrence of a binding is separated from its later uses, allowing the representation to measure whether a name becomes easier to predict after it has been introduced. First-tokenizer-piece and continuation-piece measurements allocate surprisal back to the bytes belonging to the same identifier, reducing dependence on the number of tokenizer pieces.

Contextual-support features compare the same target tokens under two prefix lengths. Targets are evaluated in 32-token blocks with either 32 or 256 preceding source tokens. For an eligible occurrence,

$$g(R) = \text{BPB}_{\text{short}}(R) - \text{BPB}_{\text{long}}(R),$$

so a positive value means that the additional preceding context makes the identical target easier to predict. Targets near the beginning of a source unit, for which the two contexts contain no additional information, are treated as unavailable rather than assigned a zero gain.

For an eligible preceding comment, the comment measurement keeps the subsequent code target fixed and compares its BPB with the comment retained and removed. One feature averages this gain over eligible comment–code pairs. The second sums the supplied prediction bits and normalises them by the complete source size, producing a whole-source density. Source units without an eligible comparison have a missing pairwise gain; their source-density value is structurally zero because no measured prediction bits are supplied by comments.

Complete-source scoring uses overlapping 512-token windows with a stride of 384 tokens while assigning each original token a single loss. Values requiring a syntactic role or a valid matched comparison are recorded as missing when that evidence is absent. Missing-value imputation is performed later within each predictor-training partition as described in the main paper’s evaluation section.

B Feature Selection Details

Feature selection consists of candidate eligibility, stability ranking within each pretrained-model instantiation, and consensus ranking across independent screens. The embedding-only and extended passes contain 181 and 228 eligible candidates, respectively.

B.1 Candidate Eligibility

Several traditional candidates adapt measurements proposed by Dorn and Scalabrino [5, 18]. We retain them in the produced feature tables but exclude 16 measurements from screening when their interpretation depends on snippet presentation, language-specific punctuation, or an unstable clustering partition:

- three cluster-size coefficients of variation: OPTICS over the `only_identifier` and `structural_core` views, and automatically selected K-means over the `semantic_core` view;
- ten absolute vertical-position measurements for identifiers, keywords, operators, periods, comments, numeric literals, and snippets; and
- period density, period-position variation, and DFT-based comma periodicity.

Only DFT-based comma periodicity would enter the first 11 positions of the unrestricted extended ranking. It remains excluded because it depends on the line-wise distribution of a language-specific punctuation mark. The policy removes three embedding-derived and thirteen traditional candidates, leaving

$$(102 - 3) + (95 - 13) = 99 + 82 = 181$$

eligible features in the embedding-only pass. All 47 causal-LM candidates are eligible, giving

$$99 + 82 + 47 = 228$$

features for each extended causal-LM–embedding-model matrix. The 99 embedding-derived columns are recomputed for each embedding model, and the 47 causal-LM columns are recomputed for each causal LM.

B.2 Stability-based Feature Ranking

For each screened matrix, we independently rank either 181 or 228 eligible candidates using an L1-regularised logistic regression [21]. Readability scores are binarised independently within every dataset at the dataset-specific median: samples with scores greater than or equal to the median are assigned to the higher-readability class. Median ties remain in that class, and no middle-score samples are discarded. This produces a common screening task over all 1,100 human-rated samples while preventing differences in numerical annotation scales from defining the screening labels.

Missing feature values are imputed using the median estimated from the corresponding fitting sample, after which the features are standardised. Samples are weighted inversely by the size of their source dataset so that each dataset contributes equal total weight. The logistic regression additionally uses balanced class weights.

The model uses $C = 0.08$, L1 regularisation, the liblinear solver, a maximum of 5,000 iterations, and random seed 42. To reduce sensitivity to sampling variation, we perform 200 stability-selection rounds [12]. In each round, 70% of the samples from each of the six datasets are selected uniformly without replacement. A new imputer, scaler, and logistic regression model are fitted to the resulting sample.

For each feature j , we compute its selection frequency π_j , defined as the fraction of completed rounds in which its coefficient satisfies

$$|\hat{\beta}_j| > 10^{-12}.$$

We additionally record the mean absolute coefficient across the stability-selection rounds and the absolute coefficient obtained by fitting the same model to the complete six-dataset screening pool.

Within each embedding-model instantiation, features are ranked lexicographically in descending order according to

$$\left(\pi_j, |\hat{\beta}_j^{\text{full}}|, \frac{1}{200} \sum_{r=1}^{200} |\hat{\beta}_j^{(r)}| \right).$$

Thus, selection frequency is the primary ranking criterion, the full-pool absolute coefficient is the first tie-breaker, and the mean absolute coefficient across stability rounds is the second tie-breaker. The canonical feature name is the final deterministic tie-breaker.

B.3 Consensus Ranking Across Pretrained Models

Each screened matrix produces an independent ranking of the same eligible feature definitions: 181 in the embedding-only pass and 228 in the extended pass. The feature matrices are not concatenated. Instead, features with the same canonical definition are matched by name. The embedding-only consensus aggregates five embedding-model rankings. The extended consensus aggregates 15 rankings obtained from the Cartesian product of three causal LMs and five embedding models.

For feature j and screened model pairing m , let π_{jm} denote its stability-selection frequency and let $\hat{\beta}_{jm}^{\text{full}}$ denote its coefficient in the complete-pool logistic-regression fit. We define the feature as active under model m when

$$a_{jm} = \mathbb{I} \left[\pi_{jm} > 0 \vee \left| \hat{\beta}_{jm}^{\text{full}} \right| > 10^{-12} \right].$$

Its cross-model support is then

$$A_j = \sum_{m=1}^M a_{jm},$$

where $M = 5$ for the embedding-only pass and $M = 15$ for the extended pass. Thus, cross-model support does not indicate membership in a separately chosen per-model Top- k set. Instead, it indicates the number of independently screened model pairings under which a feature receives nonzero support.

For every candidate feature, we additionally compute its mean selection frequency, mean absolute complete-fit coefficient, mean reciprocal rank, and best individual-model rank. The final consensus ranking orders features lexicographically according to:

- (1) cross-model support A_j (descending);

- (2) mean selection frequency across the M screened matrices (descending);
- (3) mean absolute coefficient from the M complete-pool fits (descending);
- (4) mean reciprocal rank across screened matrices (descending);
- (5) the best rank achieved under any individual screened matrix (ascending); and
- (6) canonical feature name (ascending).

The support count first distinguishes features that are consistently active across pretrained-model pairings. Because many candidates receive nonzero support from all screened matrices, the mean selection frequency provides the principal discrimination within the largest support group. The remaining criteria resolve further ties using coefficient magnitude and consistency across the per-model rankings.

The support distribution for the extended screening pool is reported in [Table 6](#). All feature families are matched by canonical definition across the 15 rankings; causal-LM values are separately measured by each causal LM.

Table 6. Distribution of the 228 eligible extended candidates by support across the 15 causal-LM–embedding-model screens.

Active screens	Candidates	Active screens	Candidates
15	133	7	4
14	6	6	7
13	10	5	4
12	15	4	1
11	5	3	5
10	10	2	2
9	3	1	2
8	5	0	16
Total		228	

In the embedding-only pass, 120 of 181 candidates receive nonzero support from all five embedding representations. In the extended pass, 133 of 228 candidates receive support in all 15 screens. Consequently, cross-model support alone is not sufficient to determine either ordering; selection frequency and the subsequent tie-breaking criteria remain important within the largest support group.

B.4 Selection Outcomes and Feature Budget

Embedding-only pass. The first pass was completed before causal-LM features were introduced. We reviewed the leading consensus-ranked candidates for interpretability, portability, redundancy, sensitivity to extreme observations, and fitted direction. Pairwise redundancy was defined by an absolute Pearson or Spearman correlation above 0.9. The resulting independently selected model contains 18 features; all occur within the first 24 consensus positions and receive nonzero support from all five embedding models. Its cross-embedding performance is reported in [Appendix F](#).

Extended pass. After adding the 47 causal-LM candidates, we rerun all 200 stability rounds for each of the 15 causal-LM–embedding-model pairings and construct a new 228-feature consensus ranking. Successive prefixes of this fixed extended ranking are evaluated in [Figure 1](#) using pooled 10-fold cross-validation on the six human-rated development datasets. At each prefix size, the plotted value for a dataset is the mean of three separately fitted predictors, one for each causal LM, using the Nomic embedding instantiation; predictions are not ensembled.

Performance rises rapidly in the leading portion of the ranking and then enters a less stable region in which additional features produce smaller, dataset-dependent changes. We retain the first 11 positions as the final operating point. The final representation is therefore the literal first 11 positions of the extended consensus ranking. A redundancy pass at the predefined absolute Pearson-or-Spearman threshold of 0.9 rejects no feature in this prefix.

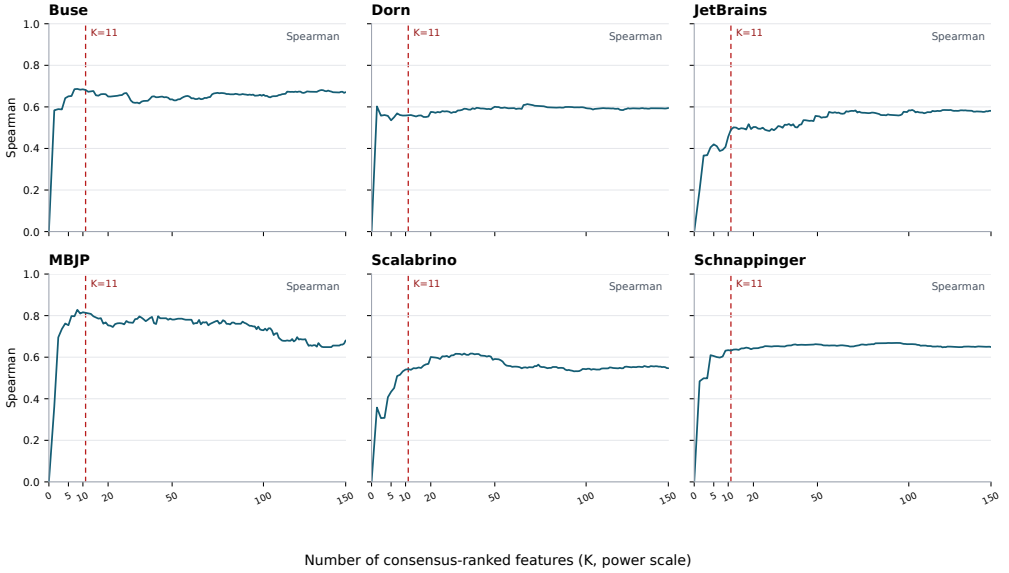


Fig. 1. Pooled 10-fold cross-validation performance for prefixes of the extended consensus ranking. The ranking is fixed before this diagnostic evaluation. At each prefix size, each curve averages the three causal-LM-specific Ridge results. The dashed line marks the final feature budget ($K = 11$).

The selected prefix contains four traditional, two embedding-derived, and five causal-LM predictability features. Every instantiation uses the same 11 canonical definitions; only the pretrained-model-dependent measurements change. The complete definitions and interpretations appear in the main paper’s approach section and [Appendix A](#).

B.5 Content-Aware Comment Diagnostic

We additionally examine whether separating comments by content yields a more useful comment-predictability signal. Comments are grouped deterministically as descriptive prose, code-like content, licence/provenance metadata, visual separators, mixed content, or unclassified content. For each group, we add its mean comment BPB as a candidate and repeat the same 15-model stability-selection procedure, expanding the candidate pool from 228 to 234 features.

Table 7. Consensus ranks of the content-aware comment-predictability candidates. Observed samples contain at least one comment in the corresponding group.

Comment content group	Observed samples	Consensus rank
Mixed content	159	23
Descriptive prose	486	43
Unclassified content	177	66
Code-like content	80	105
Visual separators	16	164
Licence/provenance metadata	193	216

None of the six candidates enters the first 11 positions, and the membership of the selected feature set remains unchanged. Separating comments by content therefore does not produce a dedicated comment-predictability feature with sufficient independent support to enter the final representation.

C Human-Rated Dataset Details

The six benchmark targets are derived from their released human ratings as follows.

C.1 Published Datasets

For Buse, Scalabrino, and Dorn, we use the arithmetic mean of the individual human ratings released with each dataset [3, 5, 18]. Buse contains 121 ratings for every snippet, while Scalabrino contains nine ratings for every method. The number of ratings in Dorn varies across snippets, ranging from 161 to 267.

Schnappinger provides an expectation-maximisation posterior over four ordered readability classes [19]. Given the posterior vector (p_0, p_1, p_2, p_3) , where the classes range from 4 (most readable) to 1 (least readable), we compute the continuous target as

$$y = 4p_0 + 3p_1 + 2p_2 + p_3.$$

For JetBrains [20], we use the fraction of votes that classify a snippet as readable:

$$y = \frac{N_{\text{readable}}}{N_{\text{readable}} + N_{\text{unreadable}}}.$$

The number of votes per JetBrains sample ranges from 11 to 31, with a median of 20. Although the dataset also supports a binary majority-vote label, we retain the vote fraction as a continuous target for the final regression and evaluation tasks so that the degree of annotator agreement is preserved. During feature screening, this continuous target is binarised at its dataset-specific median in the same manner as the other five datasets; the released majority-vote label is not used. All targets are oriented such that larger values indicate greater readability.

C.2 MBJP Development Dataset

MBJP is a small development dataset constructed specifically for this study. We select programming tasks from the Java portion of MBXP [2] and use gpt-4.1-nano-2025-04-14 to generate candidate Java solutions. The solutions are generated under two prompting conditions intended to produce contrasting readability characteristics while preserving functional correctness.

The high-readability condition asks the model to act as a proficient Java developer and produce a clear implementation:

High-readability generation prompt

You are a proficient Java developer. Generate highly readable Java code that correctly satisfies the following task. Return only the complete Java code.

Task:

{MBJP task description}

The low-readability condition asks the model to produce a correct but deliberately difficult-to-read implementation. The prompt identifies confusing control flow, ambiguous identifiers, and inappropriate comments as representative sources of low readability:

Low-readability generation prompt

You are a Java learner. Generate low-readability Java code that correctly satisfies the following task. The code must remain functionally correct.

Low-readability code may contains:

- confusing or unnecessarily control flow;
- strange or meaningless variable names;
- other characteristics that make the code harder to read.

Task:

{MBJP task description}

Each generated program is checked using the task-specific validation procedure provided by MBJP. Programs that fail validation are discarded rather than repaired or assigned a readability score. After this filtering step, the final development dataset contains 17 functionally correct Java solutions: ten generated under the high-readability condition and seven under the low-readability condition.

The prompting condition is used only to introduce variation in the generated code and is not treated as the readability ground truth. Instead, each retained solution is independently evaluated by two human annotators on a five-point readability scale. The mean of the two ratings is used as the continuous target. The annotators achieve a Spearman correlation of 0.829 and an ICC(2,k) of 0.899, indicating strong agreement [10].

D Baseline Implementations

We compare BTTF against three conventional readability baselines (LLOC, Posnett, and Scalabrino), two reconstructed learned baselines (Dorn and Mi), and three direct LLM-based baselines. Fixed published models are evaluated using their released implementations or coefficients, whereas the reconstructed baselines are trained according to the protocols described below.

D.1 Conventional and Reconstructed Baselines

Logical lines of code (LLOC) is the number of language-aware statement and declaration nodes in the source AST; comments and blank lines are not counted. Java and Python use their respective grammars, while C, C++, and CUDA share the C++ grammar. For Posnett et al. [16], we use the published model without retraining:

$$z = 8.87 - 0.033V + 0.40L - 1.5H, \quad P(\text{readable}) = \frac{1}{1 + e^{-z}},$$

where V is Halstead volume, L is the number of source lines, and H is source-code byte entropy. We evaluate the comprehensive model of Scalabrino et al. [18] using the authors' released Java

archive and pretrained classifier. Method-level Java snippets are placed in a minimal enclosing class when required by the released tool, and samples that cannot be processed are omitted rather than assigned fallback predictions.

Because no pretrained Dorn model was released, we reconstruct the model of Dorn and Weimer [5] using the public Dorn feature data and the released implementations of its visual metrics. Metrics defined for fewer than 80% of the training samples are excluded, seven features are selected through a forward wrapper evaluated with stratified 10-fold accuracy, and a logistic-regression model is fitted on all 360 Dorn samples. The readable-class probability is used as the continuous readability score.

We also reconstruct the character-level ConvNetCR branch described by Mi et al. [13]. This reproduction follows the published convolutional architecture but is not presented as the complete DeepCRM model, whose full implementation and trained weights were not released. The model contains three line-spanning convolution banks with 100 feature maps per bank, followed by ReLU activation, global max pooling, dropout, and a two-class output layer. It is trained on the highest and lowest readability quartiles of Buse, the Java subset of Dorn, and Scalabrino, with the middle half of each score distribution excluded. The resulting readable-class probability is used as its continuous score.

D.2 Direct LLM Baselines

Ouédraogo et al. [15] systematically compare zero-shot, few-shot, chain-of-thought, and tree-of-thought prompting for code readability assessment. In their study, zero-shot prompting obtains the lowest mean absolute error with respect to human readability scores. We therefore adopt their developer-guided zero-shot formulation as our direct LLM baseline and instantiate it with DeepSeek-V4-Pro-0813, GPT-6 Sol, and GPT-6.1 Sol. The main paper reports DeepSeek-V4-Pro-0813 and GPT-6.1 Sol; the complete GPT-6 Sol results are retained in the supplementary material.

Direct LLM Readability Prompt

You are a software engineer evaluating code readability.

Evaluate the following code on a scale from 0 (unreadable) to 20 (highly readable), explicitly using these criteria:

1. Code Structure - logical organisation vs tangled code
2. Nesting - flat vs deeply nested
3. Clarity of intent - clear purpose vs ambiguous
4. Code length - concise vs unnecessarily long
5. Action granularity - one action per line vs multiple
6. Reading flow - natural vs disorganised

Return ONLY JSON:

```
{ "score": "0-20",
  "reasoning": "concise explanation (max 200 words)" }
```

Code:

```
{code}
```

The same prompt is applied to every sample without demonstrations or access to the corresponding human readability labels. Each model scores every sample three times, and we report the mean. GPT-6 Sol and GPT-6.1 Sol are evaluated through the OpenAI API using the model identifiers gpt-6-sol and gpt-6.1-sol. GPT-6 uses no additional reasoning effort, while GPT-6.1 uses low reasoning effort, the minimum level supported by that model. None of the models receives

tools or executes an agentic workflow; each source unit is judged independently from the fixed prompt. The experiments were run on October 2–3, 2026, and the returned model identifiers, scores, explanations, token usage, and request parameters are retained with the per-sample results.

D.3 Direct-LLM Responses for the Buse/66 Case

Section 4.4 examines a snippet that receives a mean human rating of 2.603/5 but substantially higher scores from all three direct LLM baselines. Representative returned explanations are reproduced below; the headings give the score accompanying each displayed response.

DeepSeek-V4-Pro-0813 (16/20).

The code is generally clean and readable, with a logical structure and clear intent. It performs three distinct actions: selecting a row, retrieving a program object, and creating a context menu. Each action is on its own line, which supports good action granularity. Nesting is flat, and the flow is natural. However, the readability is slightly reduced by the long method call to create a context menu, which could be more concise or broken down. Additionally, the use of `getValueAt(row, 1)` assumes knowledge of the table model's column indices, which could be made clearer with a named constant. Overall, the code is straightforward but could benefit from minor refactoring for improved clarity.

GPT-6 Sol (18/20).

The three statements follow a logical sequence: select a row, retrieve its Program, then create a context menu. Nesting is absent, each line performs one action, and the code is concise. Intent and reading flow are clear, though the long chained call and numeric column index slightly reduce readability.

GPT-6.1 Sol (18/20).

Code Structure: Three logically ordered statements, clearly separated. Nesting: Completely flat, with no nested control flow. Clarity of intent: Selecting a row, retrieving its Program, and creating a context menu are apparent, though the variable name `p`, numeric column indices, and boolean arguments lack descriptive context. Code length: Concise, with no unnecessary statements. Action granularity: Each line performs one main action, although the final statement chains several calls. Reading flow: Natural progression from selection to data retrieval to menu creation; the long final line slightly reduces readability.

E Controlled Readability Interference Datasets

E.1 Construction and Scope

We construct two controlled readability-interference datasets, one for Java and one for Python. Each dataset contains 100 production-code classes selected evenly from four open-source projects. The Java dataset contains 25 classes from Apache Kafka, Google Guava, Netty, and Spring Framework. The Python dataset contains 25 classes from Django, Flask, Requests, and attrs.

For every source class, we independently apply 13 readability interferences. Each transformed variant is generated directly from the original source rather than from a previously transformed version. Consequently, the datasets do not define cumulative degradation levels, and the response to each interference can be evaluated against the same original source.

Each language-specific dataset contains 100 original classes and $100 \times 13 = 1,300$ interference variants, resulting in 1,400 records per language and 2,800 records in total. Some transformations are not applicable to every class. For example, integer-literal encoding cannot change a class containing no eligible decimal integer literal. Such unchanged variants remain in the dataset for completeness. The evaluation includes only source pairs that differ, yielding 896 changed Java variants and 920 changed Python variants.

Table 8. Independently applied readability interferences. “Changed” denotes the number of transformed classes whose source differs from the corresponding original.

Category	Interference	Definition	Java	Python
Comments	Comment removal	Removes comments and documentation, including Javadoc in Java and docstrings in Python.	90	84
Identifiers	Identifier shortening	Deterministically shortens eligible method, parameter, and local-variable names while avoiding naming conflicts.	98	95
Identifiers	Misleading identifiers	Replaces eligible names with length-matched, meaningful but contextually unrelated identifiers.	98	96
Identifiers	Garbled identifiers	Replaces eligible names with deterministic ProGuard-style or minifier-style short identifiers.	98	96
Expressions	Integer-literal encoding	Rewrites eligible decimal integer literals using equivalent hexadecimal notation.	38	30
Expressions	Condition inversion	Negates eligible conditions and exchanges the corresponding branches.	50	62
Code injection	Dead-branch injection	Inserts deterministic runtime-dead branches into eligible callable bodies.	86	95
Code injection	Readable unrelated-code injection	Inserts one of ten readable but functionally unrelated code templates into an eligible callable.	86	95
Code injection	Low-readability unrelated-code injection	Inserts one of ten functionally unrelated templates written with deliberately poor readability.	86	95
Layout	Partial layout compaction	Deterministically removes approximately 40% of eligible line-break groups while retaining part of the original visual structure.	100	91
Data flow	Numeric-intermediate introduction	Replaces eligible integer literals with the sum of two newly introduced short-named local variables whose values reconstruct the original literal.	28	28
Data flow	Intermediate-variable inlining	Removes eligible single-use local variables and substitutes their parenthesised initializer expressions at their sole use sites.	17	28
Control flow	Loop lowering	Rewrites eligible for loops into behaviourally corresponding while-based forms.	21	25
Total changed variants			896	920

E.2 Transformation Details

The 13 interferences cover seven readability dimensions: comments, identifiers, expressions, code injection, layout, data flow, and control flow. The transformations and the number of source classes for which each produced a source-level change are summarised in Table 8.

The two unrelated-code injection conditions use separate sets of ten templates. Readable templates use conventional control flow, formatting, and descriptive identifiers, whereas low-readability templates use compact identifiers, opaque expressions, and deliberately less transparent control flow. Both transformations insert at most one template into a class, allowing their effects to be evaluated under comparable target cardinality.

Identifier transformations operate on eligible method, parameter, and local-variable bindings and update recognised references consistently. Constructors and language-required names are preserved. Candidate names that would conflict with existing bindings or language keywords are

rejected. Misleading identifiers are selected primarily by length proximity to the original name, while garbled identifiers follow a deterministic shortest-available-name strategy.

Integer-literal encoding rewrites eligible decimal integer literals using lowercase hexadecimal notation. It does not recursively transform literals that are already hexadecimal. Numeric-intermediate introduction performs the complementary data-flow modification: an eligible integer literal is decomposed into two values stored in newly introduced short-named variables, and the original literal is replaced by their sum.

Intermediate-variable inlining targets eligible local variables that have an initializer, are not subsequently reassigned, and have exactly one supported use. The declaration is removed, and the use is replaced by a parenthesised copy of the initializer. Language-specific exclusions are applied where the transformation cannot be performed reliably.

Condition, data-flow, and control-flow transformations are applied only when the corresponding syntactic structures satisfy their safety constraints. Java loop lowering covers eligible classic and enhanced for loops. Python loop lowering covers eligible synchronous for loops using explicit iterator-based control flow. Therefore, the number of changed variants differs across transformations and languages.

E.3 Dataset Records and Validation

Both datasets are generated deterministically using the fixed seed 20260823. Every original and generated Java source is decoded as UTF-8 and parsed with Tree-sitter Java without syntax errors. Every original and generated Python source is valid UTF-8 and must parse successfully using the standard-library `ast.parse` implementation. A validation failure terminates dataset construction rather than retaining an invalid variant.

For each record, we store its source project and path, base-sample identifier, variant identifier, parent identifier, interference category, interference name, transformation statistics, line count, original content hash, and transformed content hash. Repository revisions are recorded where available. The hashes identify unchanged variants without requiring manual source comparison and support verification of deterministic regeneration.

The transformations operate locally on complete source classes. Binding-aware identifier updates, collision checks for introduced names, and structure-specific rewrite rules preserve source consistency, while parsing ensures class-level syntactic validity.

E.4 Fine-Grained Response Results

Response rates for all 13 individual interference types in Java and Python are reported in [Tables 9](#) and [10](#).

Table 9. Fine-grained response to independently applied readability interferences in Java. DeepSeek denotes DeepSeek-V4-Pro-0813; GPT-6 and GPT-6.1 denote GPT-6 Sol and GPT-6.1 Sol.

Interference	Dorn	Posnett	Scala-brino	Mi	DeepSeek	GPT-6	GPT-6.1	BTTF
Comment removal	90.0%	18.9%	84.1%	46.7%	80.0%	91.1%	91.1%	94.4%
Identifier shortening	61.2%	36.7%	16.5%	22.4%	95.9%	95.9%	96.9%	88.8%
Misleading identifier substitution	42.9%	43.9%	54.6%	38.8%	100.0%	99.0%	100.0%	96.9%
Garbled identifier substitution	63.3%	18.4%	0.0%	23.5%	100.0%	100.0%	100.0%	86.7%
Integer-literal encoding	18.4%	26.3%	42.1%	31.6%	60.5%	31.6%	42.1%	94.7%
Condition inversion and branch exchange	100.0%	28.0%	85.7%	28.0%	98.0%	98.0%	100.0%	90.0%
Dead-branch injection	100.0%	40.7%	97.6%	58.1%	100.0%	100.0%	100.0%	95.3%
Readable unrelated-code injection	98.8%	40.7%	84.7%	9.3%	100.0%	100.0%	100.0%	100.0%
Low-readability unrelated-code injection	100.0%	40.7%	67.1%	53.5%	98.8%	100.0%	100.0%	100.0%
Partial layout compaction	96.0%	49.0%	96.9%	98.0%	100.0%	100.0%	100.0%	99.0%
Numeric-intermediate introduction	100.0%	21.4%	46.4%	35.7%	92.9%	92.9%	100.0%	89.3%
Intermediate-variable inlining	88.2%	0.0%	29.4%	29.4%	70.6%	52.9%	64.7%	70.6%
For-to-while lowering	81.0%	9.5%	85.0%	14.3%	85.7%	100.0%	95.2%	81.0%

ScalaBrino produced finite predictions for 97 of the 100 Java groups, while all other methods processed all 100 groups. Percentages are calculated over each method’s available changed pairs.

Table 10. Fine-grained response to independently applied readability interferences in Python. DeepSeek denotes DeepSeek-V4-Pro-0813; GPT-6 and GPT-6.1 denote GPT-6 Sol and GPT-6.1 Sol.

Interference	Dorn	Posnett	Scala-brino	Mi	DeepSeek	GPT-6	GPT-6.1	BTTF
Comment removal	78.3%	40.5%	–	38.1%	82.1%	77.4%	88.1%	92.9%
Identifier shortening	58.5%	47.4%	–	27.4%	88.4%	84.2%	92.6%	75.8%
Misleading identifier substitution	30.5%	46.9%	–	33.3%	91.7%	89.6%	97.9%	85.4%
Garbled identifier substitution	52.6%	38.5%	–	27.1%	96.9%	88.5%	94.8%	80.2%
Integer-literal encoding	44.8%	13.3%	–	10.0%	73.3%	33.3%	53.3%	100.0%
Condition inversion and branch exchange	52.5%	41.9%	–	11.3%	95.2%	98.4%	96.8%	90.3%
Dead-branch injection	98.9%	58.9%	–	24.2%	96.8%	96.8%	100.0%	95.8%
Readable unrelated-code injection	74.5%	58.9%	–	32.6%	83.2%	89.5%	92.6%	96.8%
Low-readability unrelated-code injection	97.9%	58.9%	–	30.5%	94.7%	94.7%	95.8%	97.9%
Partial layout compaction	75.6%	56.0%	–	82.4%	90.1%	87.9%	92.3%	85.7%
Numeric-intermediate introduction	55.6%	25.0%	–	14.3%	78.6%	96.4%	96.4%	82.1%
Intermediate-variable inlining	85.2%	3.6%	–	3.6%	78.6%	53.6%	71.4%	67.9%
For-to-while lowering	62.5%	20.0%	–	16.0%	96.0%	96.0%	100.0%	60.0%

ScalaBrino is omitted from the Python experiment because its released implementation supports Java only. It produced finite predictions for 97 of the 100 Java groups. The reconstructed Dorn extractor processed 99 of the 100 Python groups. Percentages for these methods are calculated over their available changed pairs.

F Embedding-Only Model Results

Before adding causal-LM predictability candidates, the same screening procedure was applied to the 181 eligible traditional and embedding-derived candidates. The resulting independently selected model contains ten traditional and eight embedding-derived features. Its selection procedure is reported in Appendix B.

The retained measurements are operator density, source-text entropy, expression–literal density, longest line length, computation-and-control pattern count, identifier embedding dispersion, mean indentation change, structural-pattern mixing under OPTICS and agglomerative clustering, structural embedding dispersion, regular-expression prevalence, short-identifier prevalence, short-identifier mathematical context, bitwise-operation prevalence, unclustered structural-snippet ratio, identifier-length variation, decision density, and operator-character share.

Performance of the embedding-only model under each of the five embedding representations used during feature selection is reported in Table 11. The same 18 feature definitions, Ridge pipeline, datasets, and folds are used throughout; only the embedding-derived feature values change.

Table 11. Performance of the independently selected 18-feature embedding-only model across embedding representations. Each value is the unweighted average of the six dataset-specific Spearman correlations.

Embedding model	Pooled CV ρ	LODO ρ
Nomic Embed Text v1.5	0.562	0.553
Qwen3 Embedding 0.6B	0.562	0.554
Jina Embeddings v2 Base Code	0.551	0.548
Snowflake Arctic Embed M v2.0	0.551	0.561
Voyage 4 Nano	0.562	0.538

The pooled averages range from 0.551 to 0.562, and the LODO averages range from 0.538 to 0.561. The embedding-only model therefore exhibits similar aggregate performance across the five independently fitted embedding representations.

G Model Robustness and Benchmark Uncertainty

G.1 Robustness Across Pretrained-Model Combinations

We instantiate the fixed final feature schema using every pairing of the three causal code language models and five embedding models used during consensus selection. The 11 canonical feature definitions, Ridge pipeline, six datasets, and fold assignments remain fixed, and each of the 15 combinations is fitted independently.

Table 12. Performance across all 15 causal-LM–embedding-model combinations. Values are unweighted means of the six dataset-specific Spearman correlations in Panels A and B. Panels C and D report the percentage of changed variants assigned a lower readability score than their matched originals. Nomic, Qwen3, Jina, Arctic, and Voyage denote the five embedding models.

Panel A: Pooled 10-fold cross-validation					
Causal language model	Nomic	Qwen3	Jina	Arctic	Voyage
Qwen2.5-Coder-0.5B	0.613	0.624	0.620	0.616	0.613
DeepSeek-Coder-1.3B	0.625	0.629	0.626	0.633	0.623
OpenCoder-1.5B	0.622	0.626	0.631	0.629	0.624
Panel B: Leave-one-dataset-out evaluation					
Causal language model	Nomic	Qwen3	Jina	Arctic	Voyage
Qwen2.5-Coder-0.5B	0.624	0.622	0.625	0.628	0.617
DeepSeek-Coder-1.3B	0.621	0.629	0.631	0.633	0.625
OpenCoder-1.5B	0.630	0.629	0.638	0.637	0.627
Panel C: Java controlled-interference response					
Causal language model	Nomic	Qwen3	Jina	Arctic	Voyage
Qwen2.5-Coder-0.5B	95.8%	94.8%	95.1%	94.4%	95.6%
DeepSeek-Coder-1.3B	95.4%	95.3%	95.2%	94.9%	95.4%
OpenCoder-1.5B	94.8%	94.3%	93.8%	93.6%	95.2%
Panel D: Python controlled-interference response					
Causal language model	Nomic	Qwen3	Jina	Arctic	Voyage
Qwen2.5-Coder-0.5B	85.8%	81.4%	84.7%	82.8%	84.8%
DeepSeek-Coder-1.3B	84.7%	79.7%	83.5%	80.7%	83.0%
OpenCoder-1.5B	88.0%	84.2%	87.6%	85.5%	86.8%

All 15 independently fitted predictors retain the intended direction for all 11 features.

G.2 Bootstrap Confidence Intervals

We quantify uncertainty in the final OpenCoder and Jina-embedding instantiation using a dataset-stratified nonparametric bootstrap. For each of 10,000 replicates, samples are drawn with replacement within each dataset, Spearman correlation is recomputed for each dataset, and the average gives equal weight to the six datasets. BTTF’s fitted parameters and out-of-fold predictions remain fixed; the intervals therefore quantify sampling uncertainty in the reported predictions rather than variation from retraining or feature selection.

Table 13. Spearman correlations with percentile-bootstrap 95% confidence intervals for BTTF. The wider MBJP interval reflects its sample size of 17; the average intervals combine all six dataset-specific correlations with equal weight.

Dataset	Pooled CV	LODO
MBJP	0.851 [0.512, 0.980]	0.869 [0.557, 0.986]
Buse	0.671 [0.541, 0.760]	0.674 [0.551, 0.761]
Scalabrino	0.549 [0.443, 0.643]	0.560 [0.453, 0.649]
Dorn	0.561 [0.477, 0.636]	0.552 [0.467, 0.627]
Schnappinger	0.625 [0.541, 0.697]	0.633 [0.549, 0.704]
JetBrains	0.531 [0.390, 0.647]	0.540 [0.398, 0.660]
Average	0.631 [0.561, 0.673]	0.638 [0.573, 0.679]

We additionally compare the fixed BTTF predictions with GPT-6.1 Sol using a paired, dataset-stratified bootstrap. Each replicate draws the same samples for both methods within each dataset and equally averages the six resulting correlation differences.

Table 14. Paired comparison with GPT-6.1 Sol over 10,000 bootstrap replicates. The difference is BTTF minus GPT-6.1 Sol in average Spearman correlation.

Protocol	BTTF	GPT-6.1	Difference [95% CI]	<i>p</i>
Pooled CV	0.631	0.537	0.094 [0.031, 0.157]	0.0032
LODO	0.638	0.537	0.101 [0.038, 0.165]	0.0008

References

[1] Mihael Ankerst, Markus M. Breunig, Hans-Peter Kriegel, and Jörg Sander. 1999. OPTICS: Ordering Points to Identify the Clustering Structure. In *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 49–60. doi:10.1145/304182.304187

[2] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, Sujan Kumar Gonugondla, Hantian Ding, Varun Kumar, Nathan Fulton, Arash Farahani, Siddhartha Jain, Robert Giaquinto, Haifeng Qian, Murali Krishna Ramanathan, Ramesh Nallapati, Baishakhi Ray, Parminder Bhatia, Sudipta Sengupta, Dan Roth, and Bing Xiang. 2023. Multi-lingual Evaluation of Code Generation Models. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=Bo7eeXm6An8>

[3] Raymond PL Buse and Westley R Weimer. 2009. Learning a metric for code readability. *IEEE Transactions on software engineering* 36, 4 (2009), 546–558.

[4] Ricardo J. G. B. Campello, Davoud Moulavi, and Jörg Sander. 2013. Density-Based Clustering Based on Hierarchical Density Estimates. In *Advances in Knowledge Discovery and Data Mining (Lecture Notes in Computer Science, Vol. 7819)*. Springer, Berlin, Heidelberg, 160–172. doi:10.1007/978-3-642-37456-2_14

[5] Jonathan Dorn. 2012. *A general software readability model*. Master’s thesis. University of Virginia. <https://web.eecs.umich.edu/~weimerw/students/dorn-mcs-paper.pdf>

[6] Michael Günther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdessalem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, Maximilian Werk, Nan Wang, and Han Xiao. 2023. Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents. *arXiv preprint arXiv:2310.19923* (2023). arXiv:2310.19923

[7] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yifan Wu, YK Li, et al. 2024. DeepSeek-Coder: when the large language model meets programming—the rise of code intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[8] Siming Huang, Tianhao Cheng, Jason Klein Liu, Weidi Xu, Jiaran Hao, Liuyihan Song, Yang Xu, Jian Yang, Jiaheng Liu, Chenchen Zhang, et al. 2025. Opencoder: The open cookbook for top-tier code large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 33167–33193.

[9] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[10] Terry K. Koo and Mae Y. Li. 2016. A Guideline of Selecting and Reporting Intraclass Correlation Coefficients for Reliability Research. *Journal of Chiropractic Medicine* 15, 2 (2016), 155–163. doi:10.1016/j.jcm.2016.02.012

[11] James MacQueen. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1. University of California Press, Berkeley, California, 281–297.

[12] Nicolai Meinshausen and Peter Bühlmann. 2010. Stability Selection. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 72, 4 (2010), 417–473. doi:10.1111/j.1467-9868.2010.00740.x

[13] Qing Mi, Jacky Keung, Yan Xiao, Solomon Mensah, and Yujin Gao. 2018. Improving Code Readability Classification Using Convolutional Neural Networks. *Information and Software Technology* 104 (2018), 60–71. doi:10.1016/j.infsof.2018.07.006

[14] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. 2024. Nomic Embed: Training a Reproducible Long Context Text Embedder. *arXiv preprint arXiv:2402.01613* (2024). arXiv:2402.01613 [cs.CL]

- [15] Wendkúuni C Ouédraogo, Yinghua Li, Xueqi Dang, Pawel Borsukiewicz, Xin Zhou, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F Bisseyandé. 2025. Human-Aligned Code Readability Assessment with Large Language Models. *arXiv preprint arXiv:2510.16579* (2025).
- [16] Daryl Posnett, Abram Hindle, and Premkumar Devanbu. 2011. A simpler model of software readability. In *Proceedings of the 8th working conference on mining software repositories*. 73–82.
- [17] Peter J. Rousseeuw. 1987. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65. doi:10.1016/0377-0427(87)90125-7
- [18] Simone Scalabrino, Mario Linares-Vásquez, Rocco Oliveto, and Denys Poshyvanyk. 2018. A comprehensive model for code readability. *Journal of Software: Evolution and Process* 30, 6 (2018), e1958.
- [19] Markus Schnappinger, Arnaud Fietzke, and Alexander Pretschner. 2020. Defining a software maintainability dataset: collecting, aggregating and analysing expert evaluations of software maintainability. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 278–289.
- [20] Agnia Sergeyuk, Olga Lvova, Sergey Titov, Anastasiia Serova, Farid Bagirov, Evgeniia Kirillova, and Timofey Bryksin. 2024. Reassessing java code readability models with a human-centered approach. In *Proceedings of the 32nd IEEE/ACM international conference on program comprehension*. 225–235.
- [21] Robert Tibshirani. 1996. Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)* 58, 1 (1996), 267–288. doi:10.1111/j.2517-6161.1996.tb02080.x
- [22] Voyage AI. 2026. The Voyage 4 Model Family: Shared Embedding Space with MoE Architecture. <https://blog.voyageai.com/2026/01/15/voyage-4/> Official Voyage AI release announcement.
- [23] Puxuan Yu, Luke Merrick, Gaurav Nuti, and Daniel Campos. 2024. Arctic-Embed 2.0: Multilingual Retrieval Without Compromise. *arXiv preprint arXiv:2412.04506* (2024). arXiv:2412.04506
- [24] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025). arXiv:2506.05176